INNOVATIVE EVALUATION PRACTICES FOR

BCSE308P — COMPUTER NETWORKS LAB

FALL SEMESTER 2026–2027

SELF-SUPERVISED ENCRYPTED TRAFFIC CLASSIFICATION UNDER NETWORK DISTRIBUTION SHIFTS

P. SASIKIRAN — 24BCE0349

G. MONISH — 24BCE2050

A. HARSHA — 24BCE2292

B.Tech. in Computer Science and Engineering

School of Computer Science and Engineering

Vellore Institute of Technology

October 2026

ABSTRACT

Encrypted traffic classification aims to infer application or traffic categories from flow-level metadata without inspecting encrypted payload contents. A key challenge is network distribution shift: the traffic observed in a later period may differ from the data used for training. This project evaluates a supervised temporal baseline against an autoencoder-based self-supervised representation-learning pipeline using the CIC-Darknet2020 dataset (Darknet.CSV). The local dataset used during development contained 158,616 rows and 85 columns. After excluding identifiers, timestamp, and target-label columns from the predictive inputs, 64 numeric traffic features were identified.

The controlled chronological split used records before 24 February 2016 for training and records from that date onward for testing. The fair supervised temporal baseline achieved 77.8139% accuracy and 38.5675% macro-F1. The validated seed-42 self-supervised pipeline achieved 86.4741% accuracy and 48.8054% macro-F1, improvements of 8.6602 and 10.2379 percentage points, respectively. A repeated seed-123 run achieved 86.3212% accuracy and 48.4310% macro-F1. These results are promising for the selected dataset and split, but class-wise analysis indicates weak performance on minority classes, particularly file-transfer. The project is an offline research prototype and does not establish generalization to all networks or live deployment.

Keywords: *encrypted traffic, distribution shift, CIC-Darknet2020, self-supervised learning, autoencoder, temporal evaluation, macro-F1.*

CERTIFICATE

This is to certify that the project report titled "**SELF-SUPERVISED ENCRYPTED TRAFFIC CLASSIFICATION UNDER NETWORK DISTRIBUTION SHIFTS**" is a bonafide record of the work done by the students listed below in partial fulfilment of the requirements of Innovative Evaluation Practices for BCSE308P — Computer Networks Lab during Fall Semester 2026–2027.

Student name	Register number
P. Sasikiran	24BCE0349
G. Monish	24BCE2050
A. Harsha	24BCE2292

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
	Abstract	**2**
	Certificate	**3**
1	**INTRODUCTION**	**5**
1.1	Background and motivation	5
1.2	Problem statement	5
1.3	Objectives	5
1.4	Scope and limitations	5
1.5	Mapping to course modules and lab experiment	5
2	**LITERATURE SURVEY**	**6**
2.1	Overview of existing protocol or scheme	6
2.2	Review of related work	6
2.3	Research gap and justification of novelty	6
3	**PROPOSED SYSTEM**	**7**
3.1	System architecture	7
3.2	Baseline mechanism description	7
3.3	Proposed mechanism: algorithm and pseudocode	7
3.4	Novelty statement	7
3.5	Assumptions and design decisions	7
4	**IMPLEMENTATION**	**8**
4.1	Tools, platform, versions and hardware	8
4.2	Testbed setup and parameters	8
4.3	Code structure and key modules	9
4.4	Challenges and solutions	9
4.5	Label filtering and dataset row accounting	9
5	**EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION**	**10**
5.1	Performance metrics and rationale	10
5.2	Scenarios and parameter values	10
5.3	Results: baseline vs proposed	10
5.4	Analysis and discussion	11
5.5	Critical evaluation: strengths, weaknesses, failure conditions	11
6	**CONCLUSION AND FUTURE WORK**	**12**
6.1	Summary of work and key findings	12
6.2	Limitations	12
6.3	Future enhancements	12
7	**REFERENCES**	**12**
	Appendix A: Important source code/configuration	**13**
	Appendix B: Screenshots and raw result tables	**13**
	Appendix C: Assignment rubric checklist	**14**
	PPT outline from supplied template	**14**

1. INTRODUCTION

1.1 Background and motivation

Encrypted communications are widely used to protect user privacy. Traditional payload inspection is limited when the payload is encrypted, so traffic analysis can instead use flow-level metadata such as packet sizes, durations, rates, and timing patterns. A classifier trained on historical traffic may perform poorly when application popularity and class proportions change over time.

1.2 Problem statement

To investigate whether self-supervised representation learning improves encrypted traffic classification when the test data come from a later period with a different class distribution, using flow-level numeric features and a chronological train/test split.

1.3 Objectives

- Prepare the Darknet.CSV dataset and identify numeric flow features.
- Exclude identifiers, timestamps, and target labels from model inputs while retaining timestamps for chronological splitting.
- Train a fair supervised temporal baseline.
- Train an autoencoder to learn representations by reconstructing input features.
- Compare accuracy, macro-F1, and class-wise F1; preserve reproducible artifacts.
- Provide a Streamlit dashboard for viewing results and launching further experiments.

1.4 Scope and limitations

The project is an offline ML evaluation using a public dataset. It does not decrypt payloads, capture live packets, or claim production readiness. Results are limited to the dataset, chosen labels, preprocessing, and temporal boundary. Minority-class performance remains weak.

1.5 Mapping to course modules and the lab experiment

Computer Networks concept	Project connection
Traffic analysis and application protocols	Uses flow-level statistics to classify application categories without payload decryption.
Network measurement and performance evaluation	Measures accuracy and macro-F1 under a temporal shift.
Experimental methodology	Uses controlled train/test periods, saved predictions, reports, and plots.
Network security and privacy context	Studies encrypted traffic metadata rather than plaintext payloads.

2. LITERATURE SURVEY

2.1 Overview of the existing protocol or scheme

The project is a machine-learning traffic classification study rather than a new network protocol. The baseline is a supervised classifier trained on earlier-period flow features and evaluated on later-period records. The proposed pipeline adds an autoencoder-based self-supervised representation-learning stage before downstream classification. The comparison is designed to examine robustness to temporal distribution shift.

2.2 Review of related work

The following table summarizes foundational and directly relevant work. Metrics are described as reported in the cited work where applicable; this project does not reuse their numerical results as its own.

S.No.	Authors, year	Technique	Metrics and tool	Merits	Gaps
1	Hinton & Salakhutdinov, 2006	Deep autoencoders for dimensionality reduction	Reconstruction and representation quality; neural-network experiments	Shows that learned low-dimensional representations can preserve useful structure	Not specific to encrypted network traffic or temporal distribution shift
2	Rezaei & Liu, 2019	Survey of machine-learning approaches for encrypted traffic classification	Review of classification methods and evaluation practices	Organizes challenges and methods in encrypted traffic analysis	General methods may still rely on stable train/test distributions
3	This project, 2026	Autoencoder-based self-supervised representation plus downstream classifier	Accuracy, macro-F1, class-wise F1; Python, PyTorch, scikit-learn	Evaluates chronologically shifted data and reports class-level performance	Single dataset and temporal boundary; minority-class performance remains low

Bibliographic details are listed in References. The dataset description and foundational method are used as background, not as direct evidence that the proposed method will generalize to all networks.

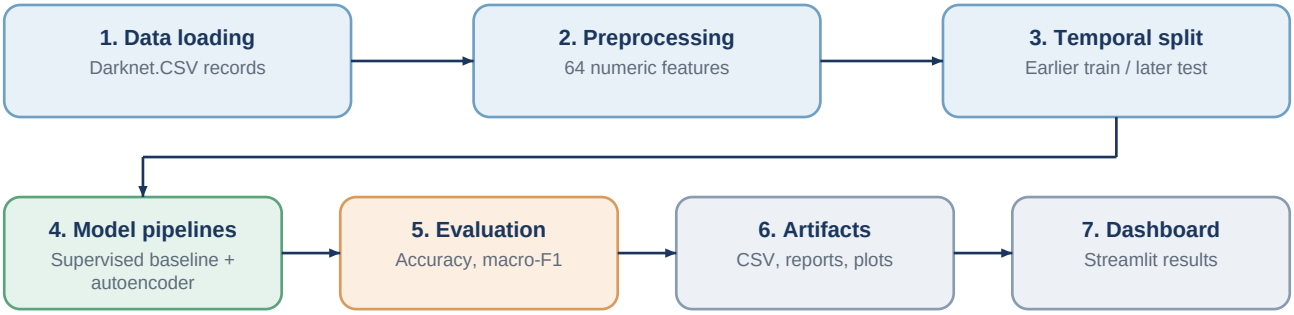
2.3 Research gap and justification of novelty

A practical gap is that random train/test splits can conceal degradation caused by time-dependent changes in traffic mix. This project explicitly uses a chronological split, compares a supervised baseline with an autoencoder-based representation pipeline, reports macro-F1 and class-wise results, and separates validated results from later dashboard experiments. The novelty is this specific experimental design and integrated reproducible workflow, not the invention of autoencoders or a new network protocol.

3. PROPOSED SYSTEM

3.1 System architecture

End-to-end architecture (offline dataset → learning → evaluation → dashboard)



Workflow: Darknet.CSV → preprocessing → chronological split → supervised baseline and autoencoder pipeline → evaluation → saved artifacts → Streamlit dashboard.

3.2 Baseline mechanism description

The fair temporal baseline is trained using the earlier time period and evaluated on the later period. It provides a reference under the same controlled split. The baseline result is 77.8139% accuracy and 38.5675% macro-F1. Predictions and report files are retained for inspection.

3.3 Proposed mechanism: algorithm and pseudocode

Algorithm: (1) Load the CSV. (2) Normalize target-label case and retain the four common classes used in the controlled experiment: audio-streaming, browsing, file-transfer, and p2p. (3) Keep the target separately and exclude identifiers, timestamps, and label columns from predictive features. (4) Split the filtered records chronologically. (5) Train the supervised baseline and the autoencoder on training features. (6) Encode train/test records, evaluate the downstream classifier, and save metrics, predictions, weights, reports, and plots.

```
Input: flow-feature table X, target y, timestamp t
Output: baseline metrics Mb, proposed metrics Mp, prediction files and model weights

Normalize labels and retain the four selected classes; then sort retained rows by t.
Define train = t < 2016-02-24 and test = t >= 2016-02-24.
Fit baseline on (Xtrain, ytrain); evaluate on (Xtest, ytest).
Train autoencoder on Xtrain; encode Xtrain and Xtest to obtain Ztrain and Ztest.
Fit/evaluate downstream classifier on the learned representations.
Save accuracy, macro-F1, class-wise F1, predictions, reports, and plots.
```

3.4 Novelty statement

The project's contribution is a controlled temporal evaluation of a self-supervised representation pipeline against a fair supervised baseline, with macro-F1 and class-wise analysis and a dashboard that keeps validated results separate from new experiments. The work does not claim a new autoencoder architecture.

3.5 Assumptions and design decisions

- The timestamp column is reliable enough to define the chronological split.
- Identifiers and target columns are not used as predictive numeric features.
- The dataset is available locally and is used according to its terms.
- Accuracy is reported alongside macro-F1 because class imbalance can make accuracy misleading.
- Only measured outputs are reported; unrun epoch configurations are not described as completed.

4. IMPLEMENTATION

4.1 Tools, platform, versions and hardware

Component	Recorded environment / role
Python	3.12.10
NumPy / Pandas	2.5.4 / 3.0.6
scikit-learn	1.9.1
PyTorch	2.14.1+cpu; CPU execution, CUDA unavailable
Streamlit	Interactive dashboard
Plotly / Matplotlib / Seaborn	Visualization and analysis
Operating environment	Windows, PowerShell, VS Code; virtual environment .venv
Dataset	Darknet.CSV; 158,616 rows × 85 columns before preprocessing

4.2 Simulation or testbed setup (topology, configuration, parameter table)

This project uses an offline dataset-based testbed rather than a packet-level network simulator. A Packet Tracer topology is not part of the implemented ML workflow.

Parameter	Recorded value
Dataset file	data/Darknet.CSV
Rows / columns before preprocessing	158,616 / 85
Numeric features	64
Training boundary	Timestamp before 2016-02-24
Testing boundary	Timestamp on/after 2016-02-24
Training rows	60,778
Testing rows	66,731
Autoencoder epochs	Validated pipeline and dashboard supports selected epoch counts; later dashboard run was 10 epochs
Execution device	CPU

4.3 Code structure and key modules

File / folder	Purpose
`app.py`	Streamlit dashboard; validated comparison, class-wise display, and experiment controls.
`train_temporal_baseline_fair.py`	Fair supervised temporal baseline and saved predictions/report.
`train_self_supervised.py`	Autoencoder-based pipeline and downstream evaluation.
`compare_models.py`	Comparison plots.
`validate_model_comparison.py`	Checks comparison artifacts.
`analyze_* / inspect_* scripts`	Explore time distribution, class shifts, and feature behaviour.
`requirements.txt`	Python dependency list.
`README.md`	Project overview, setup, dataset location, commands, results, limitations.
`results/ and screenshots/`	Saved reports, predictions, weights, plots, and interface evidence.

4.4 Challenges faced and how they were solved

Challenge	Response / mitigation
Temporal class distribution shift	Used a chronological split and reported test class counts.
Misleading aggregate accuracy	Reported macro-F1 and class-wise F1 alongside accuracy.
CPU-only environment	Trained the autoencoder on CPU and recorded the environment.
Repeatability and result confusion	Saved prediction/report artifacts and separated validated results from a later dashboard run.
Large dataset file	Kept Darknet.CSV outside the public Git repository and documented local placement.

4.5 Label filtering and dataset row accounting

The controlled four-class experiment first normalized capitalization variants in the target labels, retained the four common classes below, and then applied the chronological split. Thus, class filtering occurred before the train/test split.

Retained class	Rows	Excluded class	Rows
P2P	48,520	Chat	11,629
Browsing	46,457	Video-streaming	9,767
Audio-streaming	21,350	Email	6,145
File-transfer	11,182	VOIP	3,566
Retained: 127,509 rows		**Excluded: 31,107 rows**	
Original dataset: 158,616 rows			

The excluded 31,107 records are the Chat, Video-streaming, Email, and VOIP classes shown above. The four retained classes total 127,509 records, which are then divided chronologically into 60,778 training rows and 66,731 testing rows. Capitalization variants of the same class label were combined when counting the classes.

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance metrics and why they were chosen

Metric	Definition / reason
Accuracy	Correct predictions divided by all test predictions; intuitive but affected by class imbalance.
Macro-F1	Unweighted mean of per-class F1; gives each class equal influence.
Class-wise F1	Shows which traffic classes are difficult and reveals minority-class weaknesses.
Reconstruction loss	Tracks autoencoder input reconstruction; not a classification metric.

5.2 Scenarios and parameter values

Scenario	Description	Purpose
S1 — Random split reference	Random stratified baseline was previously measured at 91.91% accuracy and 82.21% macro-F1.	Reference only; not the main fair temporal comparison.
S2 — Controlled temporal shift	Train before 2016-02-24; test from 2016-02-24 onward.	Main evaluation of performance under a shifted class distribution.
S3 — Repeatability	Repeat self-supervised run using seed 123.	Check whether aggregate results are similar across seeds.

The random-split reference is not compared as if it were an equivalent temporal baseline; it is included to show how evaluation design changes the apparent difficulty.

5.3 Results: baseline vs proposed

Scenario	Metric	Baseline	Proposed	Improvement
S2 — temporal	Accuracy	77.8139%	**86.4741%**	**+8.6602 percentage points**
S2 — temporal	Macro-F1	38.5675%	**48.8054%**	**+10.2379 percentage points**
S3 — repeat seed 123	Accuracy	77.8139%	**86.3212%**	**+8.5073 percentage points**
S3 — repeat seed 123	Macro-F1	38.5675%	**48.4310%**	**+9.8635 percentage points**

Mean ± standard deviation was not calculated from a sufficiently large set of repeated runs. The template requests mean ± SD; the report therefore presents measured individual runs and explicitly avoids inventing a standard deviation. The seed-123 baseline value is held at the recorded fair baseline for comparison and should be interpreted as a reference, not a paired re-training run.

The proposed model was run twice (seeds 42 and 123). The mean and sample SD reported on page 11 are descriptive statistics based on only two runs (n = 2), not robust estimates of variability. The fair temporal baseline was run once, so its standard deviation cannot be calculated. The seed-123 baseline entries above reuse the single recorded baseline as a reference; they are not a separately retrained or paired baseline run.

5.4 Analysis and discussion

- The validated seed-42 self-supervised run improves accuracy by about 8.66 percentage points and macro-F1 by about 10.24 points over the fair temporal baseline.
- The seed-123 run is similar to seed 42, giving a limited repeatability signal.
- The test distribution changes sharply: P2P rises to 41,498 test examples, while file-transfer falls to 211.
- Audio-streaming F1 is approximately 0.09 and file-transfer F1 approximately 0.02 in the recorded class-wise analysis.
- The later 10-epoch dashboard run reported 86.6569% accuracy, 48.9156% macro-F1, and reconstruction loss 0.060400; it is separate from the validated comparison.

> *The proposed-model mean and sample SD are descriptive only and use two recorded runs (n = 2); two runs are insufficient for strong statistical conclusions. Only one fair-baseline run is available, so a baseline SD cannot be estimated.*

5.5 Critical evaluation: strengths, weaknesses, failure conditions

Strengths	Weaknesses / failure conditions
Chronological evaluation reflects future-period testing better than a random split.	Only one primary temporal boundary and one dataset are used.
Macro-F1 and class-wise analysis expose minority-class failures.	File-transfer and audio-streaming performance remains weak.
Saved code, reports, plots, and dashboard support review.	The dataset must be acquired separately; results can depend on preprocessing and environment.
Self-supervised pipeline improves the selected aggregate metrics.	Performance is not proven on live traffic, other networks, or new datasets.

Repeated-run summary (proposed model; two recorded seeds)

Metric	Seed 42	Seed 123	Mean ± sample SD
Accuracy	86.4741%	86.3212%	86.3976% ± 0.1081 pp
Macro-F1	48.8054%	48.4310%	48.6182% ± 0.2647 pp

> *Descriptive statistics use only two recorded proposed-model runs (n = 2); the sample SD is not a robust estimate of variability. Only one fair-baseline run is available, so baseline mean ± SD cannot be estimated. These results do not establish broad generalization.*

6. CONCLUSION AND FUTURE WORK

6.1 Summary of work and key findings

The project implemented a supervised temporal baseline and an autoencoder-based self-supervised pipeline for encrypted traffic classification under temporal distribution shift. On the validated chronological split, the baseline achieved 77.8139% accuracy and 38.5675% macro-F1, while the seed-42 proposed pipeline achieved 86.4741% accuracy and 48.8054% macro-F1. This corresponds to improvements of 8.6602 and 10.2379 percentage points. A seed-123 run achieved 86.3212% accuracy and 48.4310% macro-F1.

6.2 Limitations

- The evaluation uses one dataset and one main temporal boundary.
- The class mix changes sharply between train and test periods.
- Minority-class F1, especially file-transfer, is poor.
- No live packet capture, production deployment, or latency evaluation was conducted.
- A small number of repeated runs does not support a robust estimate of mean ± standard deviation.

6.3 Future enhancements

- Evaluate multiple temporal cutoffs and independent datasets.
- Try class-weighted losses, resampling, threshold tuning, and calibration.
- Add confusion matrices, confidence intervals, and more repeated runs.
- Explore explicit domain adaptation and more robust representations.
- Measure runtime, memory, and near-real-time performance.

7. REFERENCES

[1] G. E. Hinton and R. R. Salakhutdinov, “Reducing the Dimensionality of Data with Neural Networks,” *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[2] A. H. Lashkari, G. Kaur, and A. Rahali, “DIDarknet: A Contemporary Approach to Detect and Characterize the Darknet Traffic Using Deep Image Learning,” *Proc. 10th International Conference on Communication and Network Security (ICCNS)*, Tokyo, Japan, November 2020. Dataset: CIC-Darknet2020, Canadian Institute for Cybersecurity, University of New Brunswick. https://www.unb.ca/cic/datasets/darknet2020.html

[3] S. Rezaei and X. Liu, “Deep Learning for Encrypted Traffic Classification: An Overview,” *IEEE Communications Magazine*, vol. 57, no. 5, pp. 76–81, 2019.

[4] PyTorch, “PyTorch Documentation.” https://pytorch.org/docs/stable/index.html

[5] scikit-learn, “User Guide.” https://scikit-learn.org/stable/user_guide.html

[6] Streamlit, “Streamlit Documentation.” https://docs.streamlit.io/

[7] pandas, “pandas Documentation.” https://pandas.pydata.org/docs/

APPENDIX A: IMPORTANT SOURCE CODE / CONFIGURATION

The complete source code is maintained in the public GitHub repository. The most relevant execution commands are reproduced here for reference.

```
git clone https://github.com/palurusasikiran-netizen/encrypted-traffic-classification.git
cd encrypted-traffic-classification
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m streamlit run app.py
python train_temporal_baseline_fair.py
python train_self_supervised.py
python validate_model_comparison.py
```

Dataset placement: **`data/Darknet.CSV`**. The dataset is not embedded in the PDF or repository. The project repository contains the README, source scripts, selected validated artifacts, and eight screenshots.

APPENDIX B: SCREENSHOTS AND RAW RESULT TABLES

The repository screenshots are named as follows:

- `F2_Team28_01_Dashboard_Overview.png`
- `F2_Team28_02_Validated_Model_Comparison.png`
- `F2_Team28_03_Validated_Classwise_Performance.png`
- `F2_Team28_04_Experimental_Setup.png`
- `F2_Team28_05_Experiment_Success.png`
- `F2_Team28_06_Latest_Experiment_Results.png`
- `F2_Team28_07_Latest_Classwise_Performance.png`
- `F2_Team28_08_Latest_Experimental_Setup.png`

Run	Accuracy	Macro-F1	Additional note
Fair temporal baseline	77.8139%	38.5675%	Validated reference
Self-supervised seed 42	86.4741%	48.8054%	Validated comparison
Self-supervised seed 123	86.3212%	48.4310%	Repeat run
Later dashboard run (10 epochs)	86.6569%	48.9156%	Reconstruction loss 0.060400; separate run

APPENDIX C: ASSIGNMENT RUBRIC CHECKLIST

The following rubric is reproduced from the supplied course project template so the team can check the submission requirements.

Criterion	Marks	Evidence to prepare
Implementation and novelty (working proposed mechanism, correctness, originality)	5	Working dashboard, code walkthrough, clear novelty statement.
Performance evaluation and analysis (scenarios, repeated runs, graphs, critical interpretation, limitations)	5	Baseline/proposed tables, plots, class-wise analysis, limitations.
Demo (live baseline vs proposed, parameter change on request, time management)	5	Run dashboard and demonstrate a selected epoch setting.
Code explanation and viva (each member explains their part and answers questions)	5	Each member prepares explanation of assigned module.
Project document (format, similarity, outcome)	10	Check template compliance, references, formatting, and factual consistency.

PPT OUTLINE FROM SUPPLIED TEMPLATE

PPT outline from the supplied template: (1) Title and team; (2) Problem and motivation; (3) Existing approach and gap; (4) Proposed novelty; (5) Architecture or algorithm; (6) Implementation setup; (7) Scenarios and metrics; (8) Results; (9) Analysis and limitations; (10) Conclusion, future work, and references.